Go Fundamentals for Distributed Algorithms IV

Danilo Kaćanski

Faculty of Technical Sciences, Novi Sad
Applied Algorithms in Control Systems

March 2026.

Concurrency

Why Concurrency Matters

- **Concurrency** means multiple tasks can make progress during the same program run.
- In Go, concurrency is usually done with:
  - **goroutines** for concurrent execution
  - **synchronization tools** for coordination
- Concurrency is great, but it introduces **new classes of bugs**:
  - data races
  - deadlocks
  - livelocks
  - starvation

Key Takeaway

Concurrency improves responsiveness and structure, but shared state must be coordinated carefully.

Goroutines

Launching a goroutine

```
func sayHello() {
    fmt.Println("hello")
}
func main() {
    go sayHello()
    fmt.Println("main finished")
}
```

- `go f()` starts function `f` concurrently, but the caller does **not wait automatically**.
- Goroutines are lightweight compared to operating system threads.

Pitfall

If `main()` finishes too early, the goroutine may never complete.

Mutex I

1000 goroutines increment one counter

```
var counter int
var mu sync.Mutex
func worker() {
    mu.Lock() // enter critical section
    counter++ // shared state update
    mu.Unlock() // leave critical section
}
func main() {
    for i := 0; i < 1000; i++ {
        go worker()
    }
}
```

- All goroutines modify the same variable `counter` and `sync.Mutex` ensures only one goroutine updates it at a time.

Mutex II

- This avoids overlapping writes to shared memory.

Pitfall

Even though the counter is protected, the program can still end before all goroutines run.

Key Takeaway

Mutex solves mutual exclusion, not completion.

Why WaitGroup Is Needed

Problem: goroutines may not finish

```
func main() {
    for i := 0; i < 3; i++ {
        go worker()
    }
    fmt.Println("main finished")
}
```

- Goroutines run **asynchronously** and main does not wait.
- The program may terminate **before workers complete**.
- This can lead to missing output, incomplete work and incorrect results

Pitfall

Launching goroutines does NOT guarantee they will execute before the program exits, because of what we need a mechanism to explicitly wait for all goroutines to finish.

WaitGroup I

Synchronization pattern

```
var wg sync.WaitGroup
for i := 0; i < 1000; i++ {
    wg.Add(1) // (1) announce new work
    go func() {
        defer wg.Done() // (2) signal completion
        worker()
    }()
}
wg.Wait() // (3) wait until all are done
```

- WaitGroup internally keeps a **counter**.
- `Add(1)` → increases counter (new goroutine expected)
- `Done()` → decreases counter (goroutine finished)
- `Wait()` → blocks until counter becomes **0**

WaitGroup II

Pitfall

If the counter never reaches 0, `Wait()` will block forever.

Key Takeaway

WaitGroup ensures that the program continues only after all goroutines complete.

Why Use `defer wg.Done()` Here I

Without defer (unsafe)

```
go func() {
    worker()
    wg.Done() // might never run if something goes wrong
}()
```

With defer (safe)

```
go func() {
    defer wg.Done() // guaranteed to run
    worker()
}()
```

Why Use `defer wg.Done()` Here II

- **defer** ensures `Done()` is executed when the function ends
- Even if:
  - the function returns early
  - an error occurs
  - logic becomes more complex later
- It protects against **forgotten Done calls**

Pitfall

If `Done()` is skipped, `Wait()` will block forever, so the program hangs.

Key Takeaway

Always use `defer wg.Done()` at the start of a goroutine to guarantee completion.

Deadlock I

- A goroutine "waits" when it tries to acquire a lock that is already taken:
  - `mutex.Lock()` blocks until the lock is free
- In a deadlock:
  - Goroutine A waits for a resource held by B
  - Goroutine B waits for a resource held by A
- Neither can continue $\rightarrow$ both are stuck forever

Key Insight

"Waiting forever" means goroutines are blocked inside `Lock()` and will never proceed.

Important

This is not just slow execution, it is a complete stop of progress.

Deadlock II

- Two goroutines: `f1()` and `f2()`

Execution scenario

1. `f1()` locks `mutex1`
2. `f2()` locks `mutex2`
3. `f1()` tries to lock `mutex2` → **blocked**
4. `f2()` tries to lock `mutex1` → **blocked**

Result

- `f1()` waits for `f2()` to release `mutex2`
- `f2()` waits for `f1()` to release `mutex1`

Deadlock

Each goroutine is waiting for the other, which makes circular dependency, and result is no progress.

Deadlock III

Run this to observe deadlock

```
var m1, m2 sync.Mutex
func f1() {
    m1.Lock()
    time.Sleep(100 * time.Millisecond) // force ordering
    m2.Lock()}
func f2() {
    m2.Lock()
    time.Sleep(100 * time.Millisecond) // force ordering
    m1.Lock()}
func main() {
    go f1()
    go f2()
    time.Sleep(1 * time.Second)
    fmt.Println("done")}
```

TryLock()

Blocking vs non-blocking

```
// Blocking
mutex.Lock() // waits until lock is free
// Non-blocking
ok := mutex.TryLock() // returns immediately
```

- `Lock()` will wait until the lock becomes available
- `TryLock()` **does not wait**
  - returns `true` if lock is acquired
  - returns `false` if lock is already taken
- This allows the program to **react instead of blocking**

Pitfall

Because it does not wait, naive retry loops can create livelock.

Livelock I

Retry loop

```
for {
    if mutex1.TryLock() {
        if mutex2.TryLock() {
            break // success
        }
        mutex1.Unlock() // give up
    }
}
```

- Step 1: try to lock `mutex1`
- Step 2: if successful, try to lock `mutex2`
- Step 3: if both are acquired it is success, and if second fails, it realises first lock and retries

Key Insight

The goroutine never blocks, it just keeps retrying.

Livelock II

- Two goroutines run similar logic:
  - G1 tries: `mutex1` → `mutex2`
  - G2 tries: `mutex2` → `mutex1`

Execution pattern

1. G1 locks `mutex1`
2. G2 locks `mutex2`
3. G1 fails to lock `mutex2` → releases `mutex1`
4. G2 fails to lock `mutex1` → releases `mutex2`
5. Both retry immediately → same situation repeats

Livelock

Goroutines are active, but constantly undo each other, so there will be no progress.

Livelock III

Add delay / randomness

```
for {
    if mutex1.TryLock() {
        if mutex2.TryLock() {
            break
        }
        mutex1.Unlock()
    }
    time.Sleep(time.Millisecond) // break symmetry
}
```

Key Takeaway

Small delays or randomness allow one goroutine to succeed and break the loop.

Reader and writer methods

```
type Counter struct {
    mu sync.RWMutex
    count int
}
func (c *Counter) Get() int {
    c.mu.RLock()
    defer c.mu.RUnlock()
    return c.count
}
func (c *Counter) Inc() {
    c.mu.Lock()
    defer c.mu.Unlock()
    c.count++
}
```

When RWMutex Helps

- `RLock/RUnlock` allow multiple readers simultaneously.
- `Lock/Unlock` still gives exclusive access to writers.
- This is useful when reads happen much more often than writes.
- If writes are frequent, RWMutex may add overhead without improving performance, so this helps only if workload if read-heavy, not automatically in every case.
- Use **Mutex** when:
  - reads and writes are both common
  - simplicity matters more than optimization
- Use **RWMutex** when:
  - many goroutines mostly read
  - writes are relatively rare

Key Takeaway

RWMutex is a specialized optimization for read-dominated workloads.

Common Concurrency Mistakes

- Starting goroutines and assuming they finish before `main()` ends
- Using a Mutex but forgetting about completion (`WaitGroup`)
- Locking multiple mutexes in inconsistent order
- Retrying aggressively and creating livelock
- Holding locks too long and causing starvation

Exercise

Task 1: Counter, Mutex, and WaitGroup

Task 1

Write a program that:

- creates a shared integer counter
- launches **100 goroutines**
- each goroutine increments the counter exactly once
- uses **sync.Mutex** to protect `counter++`
- uses **sync.WaitGroup** to wait for all goroutines

Task 2: Create and Fix a Deadlock

Task 2

Write a program with:

- two mutexes: `m1` and `m2`
- two goroutines:
  - first locks `m1` then `m2`
  - second locks `m2` then `m1`
- add a short `time.Sleep(...)` between the two lock calls

Run it and observe the deadlock.

Then fix it

Change the program so that **both goroutines lock in the same order**.

Task 3: Read-Heavy Counter with Mutex vs RWMutex

Task 3

Create a `Counter` struct with one integer field and implement:

- one version protected by **sync.Mutex**
- one version protected by **sync.RWMutex**

Then:

- launch many reader goroutines calling `Get()`
- launch only a few writer goroutines calling `Inc()`
- print some outputs to confirm correctness